



**FACULTAD DE INGENIERÍA
EN ELECTRICIDAD Y COMPUTACIÓN**

Inteligencia Artificial Paralelo #1

Grupo #3

**Clasificación de egresos hospitalarios según
riesgo de fallecimiento en Ecuador usando
Machine Learning**

Integrantes:

ZAMBRANO TOALA LEONARDO THOMAS

MURILLO MORA JAVIER JOSUÉ

GUAMANÍ HERRERA JOEL ISAIÁS

Profesor:

PELÁEZ JARRIN COLÓN ENRIQUE

Fecha de entrega: 18/08/2026

Descripción

En Ecuador, los establecimientos de salud hospitalaria generan una gran cantidad de registros sobre pacientes que ingresan, reciben atención médica y posteriormente egresan de una institución de salud. Según el portal Datos Abiertos Ecuador, el Registro Estadístico de Egresos Hospitalarios 2024 contiene 1.132.667 registros sobre información de mortalidad hospitalaria reportada por establecimientos de salud [1]. Además, el INEC informa que en 2024 se registraron 1.132.667 egresos hospitalarios, 23.726 camas disponibles y 626 establecimientos de salud a nivel nacional [2].

El problema concreto que se busca abordar es la dificultad de identificar, dentro de los egresos hospitalarios en Ecuador, qué características disponibles al momento del ingreso están asociadas con un mayor riesgo de fallecimiento durante la hospitalización.

La dificultad principal está en que el fallecimiento hospitalario no puede explicarse sólo observando una sola variable. Un egreso hospitalario puede estar relacionado con la edad del paciente, sexo, lugar de residencia, fecha de ingreso y egreso, días de estadía, diagnóstico definitivo del egreso, servicio del que egresó, provincia donde fue atendido y tipo de establecimiento según INEC [3].

La condición al egreso, que permite identificar si el paciente egresó vivo o fallecido, tampoco se utiliza como característica de entrada. Esta variable se conserva únicamente como variable objetivo o etiqueta para entrenar y evaluar los modelos de clasificación.

La dificultad principal se encuentra en que el fallecimiento hospitalario no depende necesariamente de una sola característica. Factores como la edad, sexo, lugar de residencia, fecha de ingreso y características del establecimiento pueden interactuar entre sí, generando diferentes perfiles dentro de los registros hospitalarios. Debido al gran volumen de información y a la combinación de variables categóricas y numéricas, resulta necesario utilizar técnicas de Machine Learning y Deep Learning capaces de identificar patrones dentro del conjunto de datos.

Características por lo que requiere técnicas de Inteligencia Artificial, Machine Learning o Deep Learning.

Primero, existe un gran volumen de datos históricos, con más de un millón de registros [1]. Segundo, los egresos hospitalarios incluyen una condición final del paciente, como alta o fallecimiento, lo que permite distinguir casos con resultados diferentes [3]. Tercero, el problema incluye muchas variables de distinto tipo, como datos del paciente, días de

estadía, diagnóstico y servicio hospitalario [3]. Cuarto, la mortalidad hospitalaria puede depender de combinaciones de factores y no de una sola variable. Por esto, un análisis tradicional puede quedarse corto para encontrar patrones complejos dentro de la base.

Además, se debe considerar que el conjunto presenta un fuerte desbalance de clases, debido a que los fallecimientos representan aproximadamente el 1.56 % de los registros,

En consecuencia, el problema identificado es la dificultad de reconocer perfiles de egresos hospitalarios con mayor riesgo de fallecimiento dentro del sistema hospitalario ecuatoriano, usando información histórica oficial. Comprender estos patrones puede aportar al análisis de salud pública y a la gestión hospitalaria, sin que esto implique reemplazar el criterio médico ni tomar decisiones clínicas individuales de forma automática.

Aspectos específicos del problema resueltos mediante IA, ML y DL

El proyecto se enfocó en resolver la dificultad de estimar el riesgo de fallecimiento hospitalario utilizando únicamente información disponible al momento del ingreso del paciente.

En primer lugar, se realizó una separación temporal de las variables para evitar fuga de información (*data leakage*), excluyendo del predictor los datos conocidos únicamente durante o al finalizar la hospitalización, como la condición al egreso, días totales de estadía, especialidad de egreso, fecha de egreso y diagnóstico definitivo.

En segundo lugar, se trabajó con un conjunto de 1,132,667 registros con una proporción aproximada de 1.56 % de fallecimientos, por lo que fue necesario considerar explícitamente el fuerte desbalance entre las clases.

También se resolvió el tratamiento de variables heterogéneas, valores faltantes y categorías mediante técnicas de ingeniería de características, imputación, escalamiento y codificación One-Hot.

Finalmente, se compararon modelos lineales y arquitecturas de Deep Learning para determinar qué enfoque era más adecuado para identificar la clase de fallecimiento. El modelo seleccionado se integró posteriormente en una aplicación que permite realizar inferencias utilizando los mismos pasos de preprocesamiento empleados durante el entrenamiento.

Objetivo

Clasificar los egresos hospitalarios del Ecuador según su riesgo de fallecimiento hospitalario mediante variables disponibles al ingreso, utilizando técnicas de Inteligencia Artificial.

Clientes o Usuarios Potenciales

Organización	Departamento que beneficia	Cómo beneficia a la organización
Hospitales públicos y privados del Ecuador	Dirección médica	Les permitiría analizar los egresos hospitalarios con mayor riesgo de mortalidad y reconocer patrones. Con esto, podrían tener una mejor visión de los casos más delicados y apoyar la toma de decisiones internas.
Ministerio de Salud Pública del Ecuador	Área de vigilancia y salud pública	Ayudaría a identificar grupos de pacientes, provincias o tipos de diagnósticos donde existe mayor riesgo de fallecimiento hospitalario. Para planificar programas de salud, priorizar recursos y estudiar mejor la situación hospitalaria del país.
INEC u organismos de análisis estadístico en salud	Área de análisis de datos e indicadores de salud	Les permitiría aprovechar los registros de egresos hospitalarios no solo para generar estadísticas generales, sino también para encontrar patrones de riesgo en los datos. Esto puede fortalecer los informes y estudios sobre morbilidad y mortalidad hospitalaria en Ecuador.

Administradores de hospitales y responsables de calidad	Departamento de calidad y seguridad del paciente	Les ayudaría a revisar qué características aparecen con mayor frecuencia en los egresos con fallecimiento. Con esa información podrían analizar procesos internos, detectar áreas que requieren seguimiento y proponer mejoras en la gestión hospitalaria.
---	--	--

Modelos completos de análisis y definición del problema

6.1 Diagrama de caso de uso

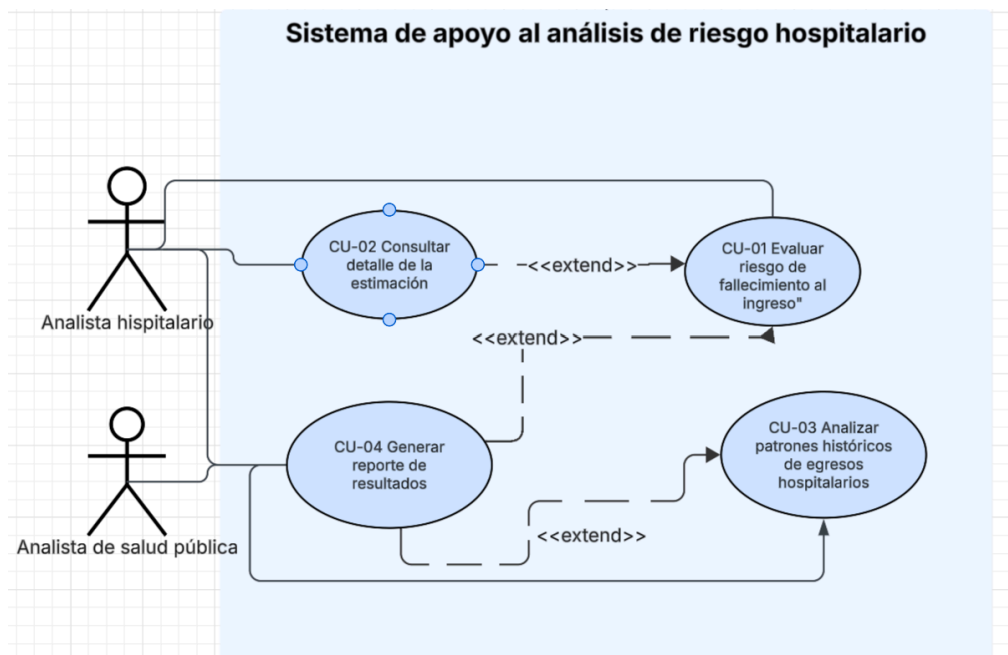


Figura 1. Diagrama de Casos de Uso del sistema de estimación de riesgo hospitalario.

Escenarios

CU-01: Evaluar riesgo de fallecimiento al ingreso

Campo	Descripción
Identificador	CU-01
Nombre	Evaluar riesgo de fallecimiento al ingreso
Actor principal	Analista hospitalario

Objetivo	Obtener una probabilidad y un nivel estimado de riesgo
Precondiciones	Disponer de las características requeridas del ingreso
Poscondiciones	Se genera una estimación o se muestra una advertencia

Escenario principal: datos completos y válidos

Paso	Acción
1	El analista selecciona la opción para evaluar el riesgo.
2	Ingresa los datos disponibles durante el ingreso.
3	El sistema verifica los campos obligatorios.
4	Se genera la estimación.
5	Se presenta la probabilidad y el nivel de riesgo.
6	Se muestra la advertencia de uso académico.

CU-02: Consultar detalle de la estimación

Campo	Descripción
Identificador	CU-02
Nombre	Consultar detalle de la estimación
Actor principal	Analista hospitalario
Objetivo	Revisar información adicional sobre el resultado
Precondiciones	Debe existir una estimación previa
Poscondiciones	Se presenta el detalle o se informa que no existe una evaluación

Escenario principal: existe una estimación

Paso	Acción
1	El analista obtiene una estimación.

2	Selecciona la opción para consultar el detalle.
3	Se muestra la probabilidad, nivel de riesgo y datos considerados.
4	Se presenta el modelo utilizado y sus limitaciones.
5	El analista revisa la información.

CU-03: Analizar patrones históricos de egresos

Campo	Descripción
Identificador	CU-03
Nombre	Analizar patrones históricos de egresos
Actor principal	Analista de salud pública
Actor secundario	Analista hospitalario
Objetivo	Consultar relaciones agregadas entre variables y condición al egreso
Precondiciones	La base histórica debe estar disponible
Poscondiciones	Se muestran resultados o una advertencia

Escenario principal: existen suficientes registros

Paso	Acción
1	El analista selecciona los filtros.
2	Se recuperan los registros correspondientes.
3	Se verifica que exista una cantidad suficiente.
4	Se generan tablas y gráficos.
5	Se muestran distribuciones de vivos y fallecidos.
6	El analista interpreta los patrones agregados.

CU-04: Generar reporte de resultados

Ficha general

Campo	Descripción
Identificador	CU-04
Nombre	Generar reporte de resultados

Actor principal	Analista hospitalario o de salud pública
Objetivo	Obtener un resumen de la estimación o del análisis histórico
Precondiciones	Debe existir una estimación o consulta previa
Poscondiciones	El reporte se genera o se informa el problema

Escenario principal: reporte generado correctamente

Paso	Acción
1	El analista obtiene una estimación o análisis histórico.
2	Selecciona generar reporte.
3	El sistema organiza los resultados disponibles.
4	Se incorporan filtros, tablas, gráficos y advertencias.
5	Se genera el reporte.

6.2 Diagrama de estado

Permite determinar qué variables pueden utilizarse para estimar el riesgo al momento del ingreso y cuáles deben excluirse por conocerse únicamente durante o después de la hospitalización. Esta separación es necesaria para evitar fuga de información.

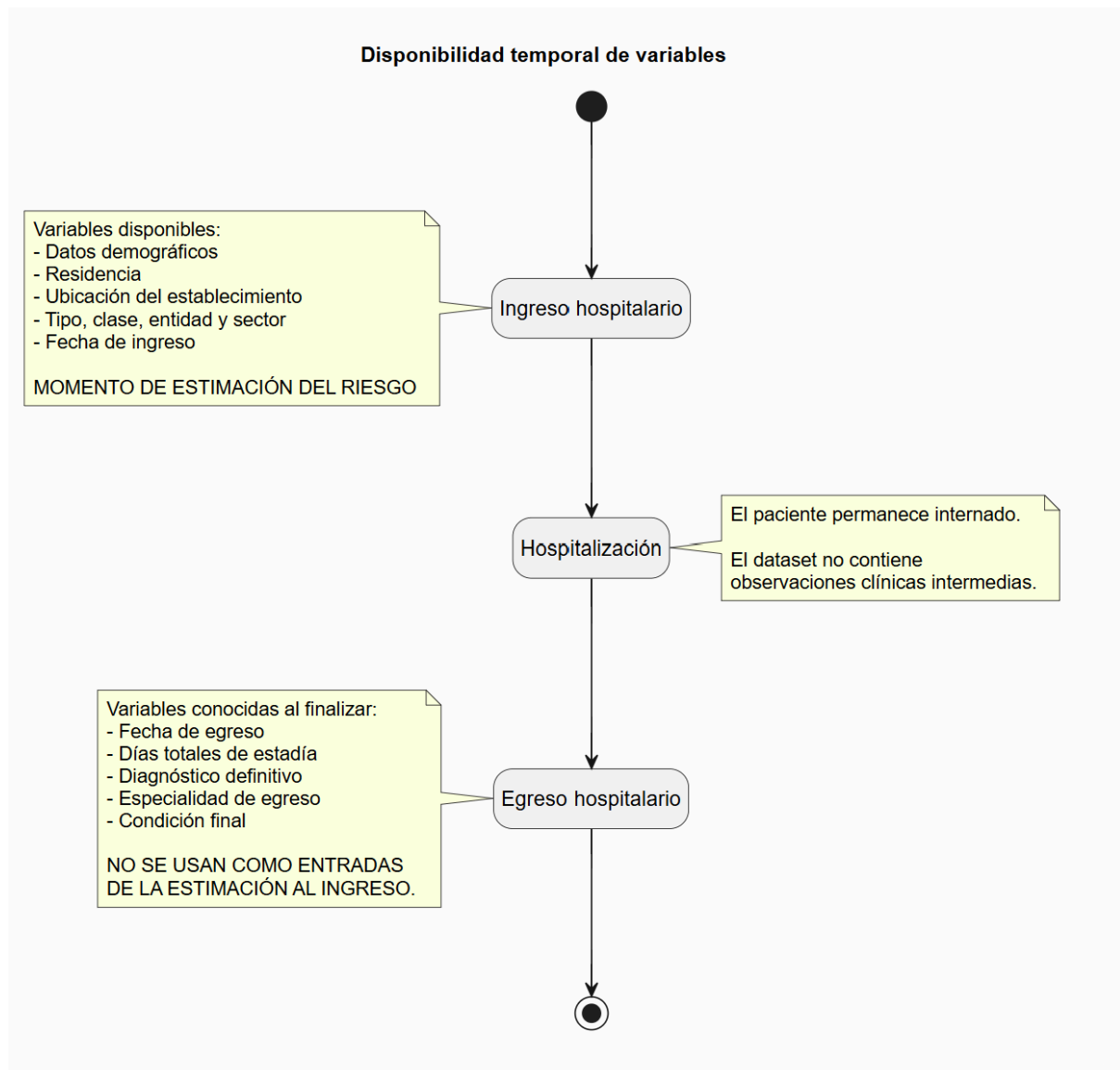


Figura 2. Diagrama de Estados y disponibilidad temporal de variables.

Durante el ingreso se encuentran disponibles las características del paciente. En este momento se realizará la estimación del riesgo.

Durante la hospitalización, el paciente permanece internado. Sin embargo, la base de datos utilizada no contiene signos vitales, resultados de laboratorio, diagnósticos provisionales ni observaciones clínicas intermedias. Por este motivo, el proyecto no puede construir una predicción dinámica que se actualice diariamente.

Al finalizar la hospitalización se registran la fecha de egreso, el total de días de estadía, el diagnóstico definitivo, la especialidad de egreso y la condición final del paciente. La condición al egreso se utilizará únicamente como etiqueta durante el entrenamiento y evaluación de los modelos.

6.2 Diagrama de actividades del problema

Describe cómo se desarrolla actualmente el problema antes de incorporar una solución basada en inteligencia artificial.

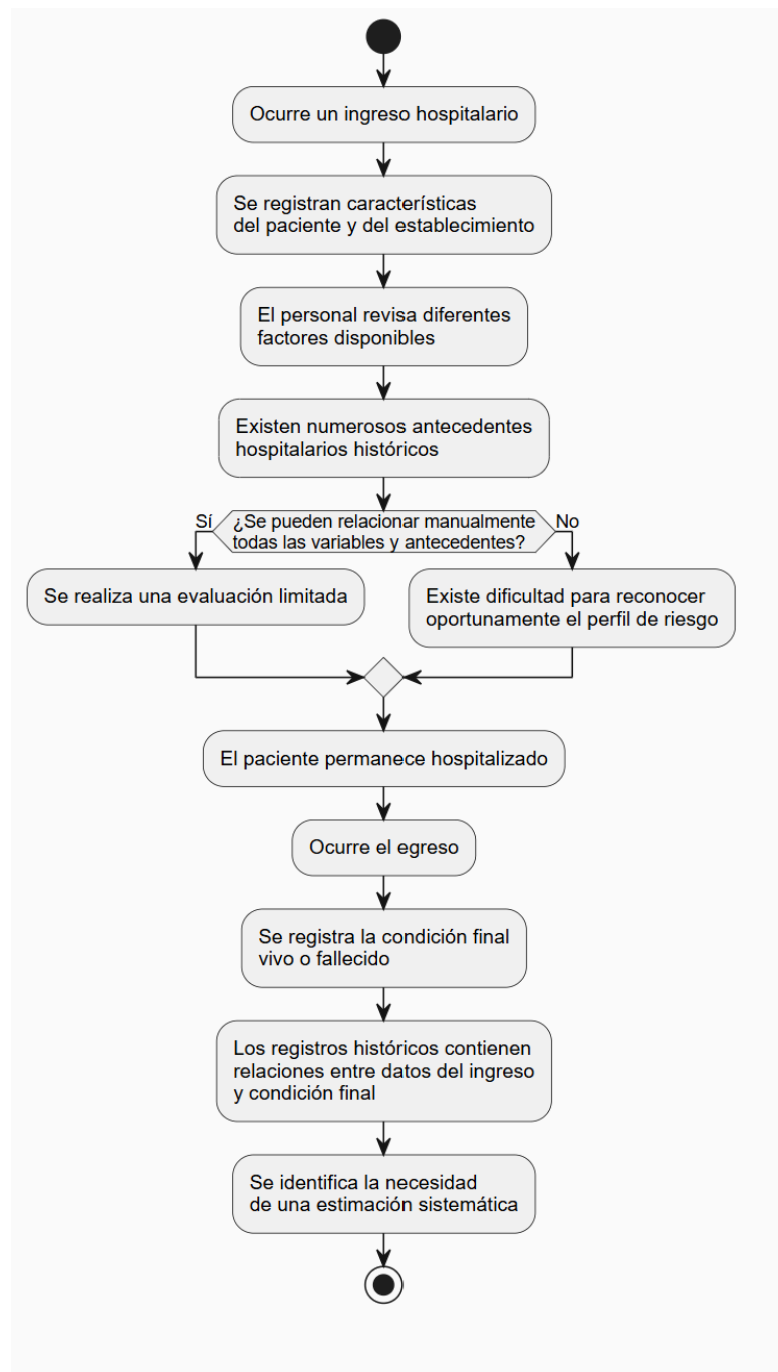


Figura 3. Diagrama de Actividades del problema de estimación del riesgo hospitalario.

El proceso comienza cuando ocurre un ingreso hospitalario y se registran las características del paciente y del establecimiento. El personal dispone de esta información para analizar la situación, pero debe considerar varios factores simultáneamente.

La dificultad surge porque existen más de un millón de registros históricos con diferentes combinaciones de edad, sexo, residencia, establecimiento, sector y condición final. Revisar manualmente todas estas combinaciones para reconocer perfiles asociados con mayor mortalidad resulta complejo.

El paciente continúa su hospitalización y, cuando ocurre el egreso, se registra su condición final como vivo o fallecido. De esta manera, los registros históricos conservan relaciones entre la información disponible al ingreso y el resultado final.

Modelos finales de diseño de la solución y técnicas de IA, ML y DL utilizadas

Descripción del diseño de la solución

La solución desarrollada consiste en un sistema de clasificación supervisada que utiliza los registros históricos de egresos hospitalarios del Ecuador para estimar el riesgo de fallecimiento mediante información disponible al momento del ingreso.

La solución incluye cinco modelos de clasificación: Regresión Logística, Perceptrón Simple, Multi-Layer Perceptron (MLP), CNN 1D y LSTM. Los modelos se entrenaron y evaluaron bajo un mismo problema de clasificación binaria, permitiendo comparar enfoques lineales y arquitecturas de Deep Learning.

Para prevenir fuga de información, se excluyeron de las entradas predictivas las variables conocidas únicamente durante o después de la hospitalización, entre ellas el diagnóstico definitivo, días totales de estadía, especialidad de egreso, fecha de egreso y condición final. La condición al egreso se utilizó únicamente como variable objetivo.

La solución también incorpora un pipeline de preprocesamiento reutilizable, un servicio backend desarrollado con FastAPI y una interfaz web que permite ingresar las variables requeridas, ejecutar una estimación, consultar el resultado, revisar estadísticas históricas y generar un reporte.

Diagrama de Clases de Diseño

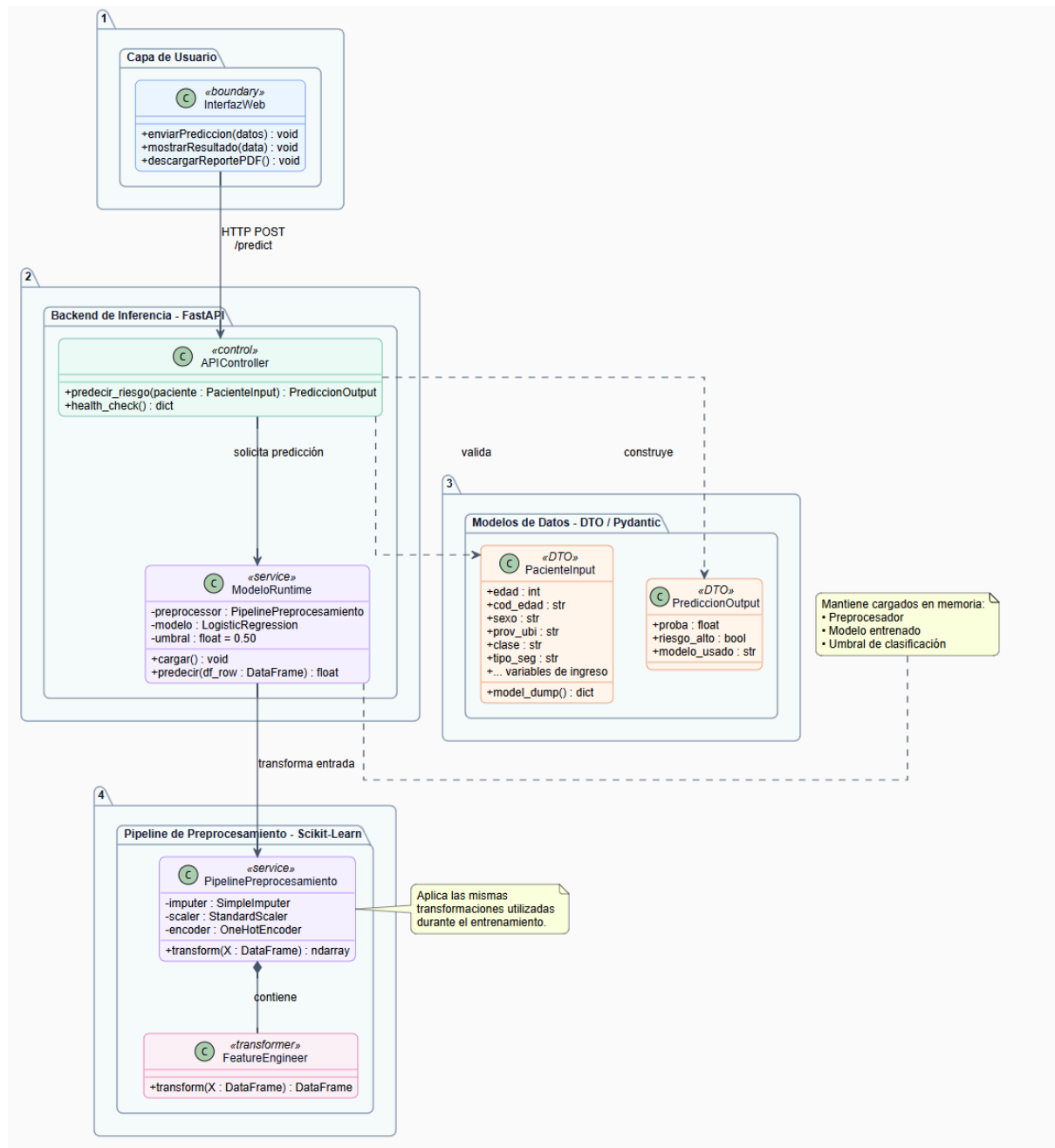


Figura 4. Diagrama de Clases de Diseño de la arquitectura de software y Machine Learning

El diagrama de clases representa la estructura principal del software utilizado para realizar la inferencia. La solución se divide en una capa de usuario, una capa de control e inferencia, los modelos de transferencia de datos y el pipeline de preprocesamiento.

Diagrama de Componentes de la Solución y del Pipeline de IA

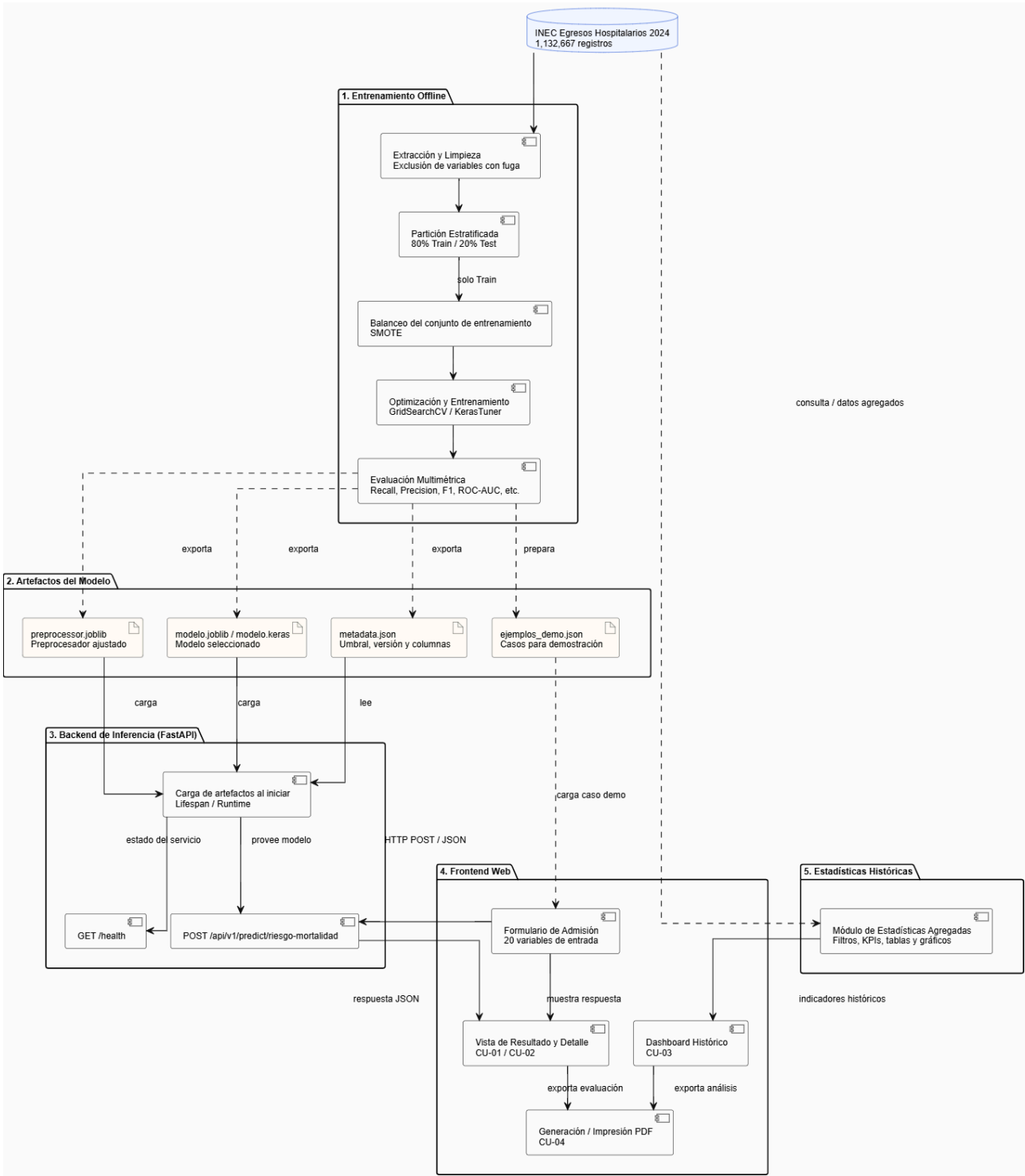


Figura 5. Diagrama de Componentes de la Solución y del Pipeline de IA

El diagrama de componentes representa la arquitectura completa de la solución, incluyendo las etapas de entrenamiento offline, almacenamiento de artefactos, servicio de inferencia,

frontend y análisis histórico. El conjunto de datos del INEC se utiliza durante la fase de entrenamiento para realizar la limpieza, preparación y entrenamiento de los modelos. Una vez seleccionado el modelo final, el preprocesador, el modelo y los metadatos necesarios se almacenan como artefactos reutilizables. El backend desarrollado con FastAPI carga estos componentes al iniciar el servicio y permite procesar nuevas solicitudes de inferencia. Finalmente, el frontend consume el servicio y presenta las funciones de estimación individual y análisis histórico.

Diagrama de Secuencias

A. Secuencia: Inferencia y Consulta de Detalle (CU-01 y CU-02)

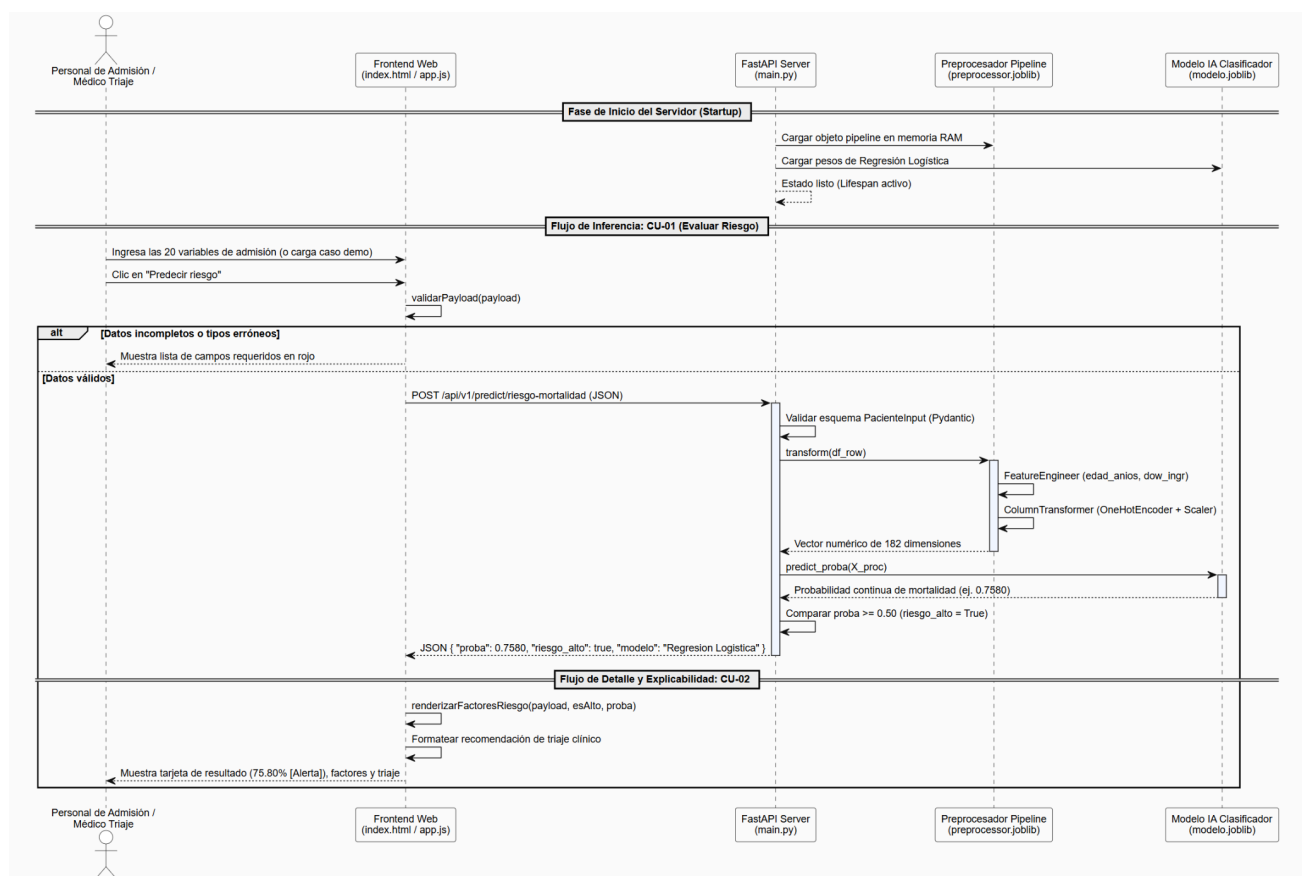


Figura 6. Diagrama de Secuencia para la Inferencia de Riesgo y Consulta de Detalle (CU-01 y CU-02)

El diagrama representa el flujo de una solicitud de inferencia desde la interfaz web hasta el modelo predictivo.

B. Secuencia: Análisis de Patrones Históricos y Reporte (CU-03 y CU-04)

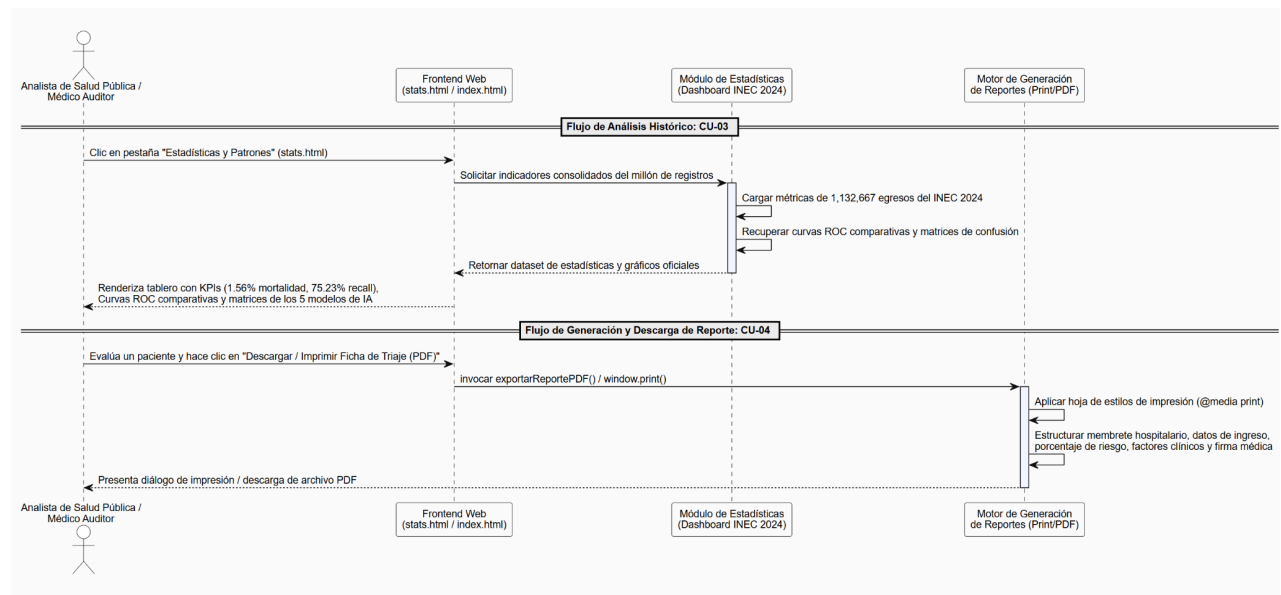


Figura 7. Diagrama de Secuencia para el Análisis Histórico y Generación de Reporte (CU-03 y CU-04)

Este diagrama representa el flujo utilizado para consultar la información histórica y generar reportes. El usuario accede al módulo de estadísticas, donde se presentan indicadores generales, distribuciones, resultados comparativos de los modelos y otros elementos visuales obtenidos de los registros históricos.

Técnicas de IA, Machine Learning y Deep Learning implementadas

Para resolver el problema de clasificación del riesgo de fallecimiento hospitalario se implementaron y compararon cinco modelos de aprendizaje supervisado con características diferentes. Se incluyeron modelos lineales y arquitecturas de redes neuronales con el objetivo de analizar su desempeño sobre el conjunto de datos y determinar cuál presenta una mejor capacidad para identificar los casos de fallecimiento.

Regresión Logística

La Regresión Logística se utilizó como modelo lineal de referencia o *baseline*. Es un algoritmo de clasificación que permite estimar la probabilidad de que un registro pertenezca a una de dos clases, por lo que resulta adecuado para problemas de clasificación binaria [4], [12].

En este proyecto, las clases representan los egresos correspondientes a pacientes vivos y fallecidos. La Regresión Logística permite establecer un punto de comparación inicial frente a modelos de mayor complejidad y determinar hasta qué punto las características utilizadas permiten separar ambas clases mediante un modelo lineal.

Perceptrón Simple

El Perceptrón es un modelo de clasificación lineal que aprende ajustando sus pesos a medida que procesa los ejemplos de entrenamiento [22]. Cuando una predicción es incorrecta, el modelo modifica estos pesos para mejorar su capacidad de clasificación en las siguientes iteraciones.

En el proyecto, este modelo se utilizó para evaluar la capacidad de separar las clases utilizando un enfoque lineal después de realizar el preprocesamiento de las características. También permite comparar un modelo relativamente sencillo con las arquitecturas de redes neuronales de mayor complejidad utilizadas posteriormente.

Multi-Layer Perceptron (MLP)

El Multi-Layer Perceptron (MLP) es una red neuronal formada por una capa de entrada, una o más capas ocultas y una capa de salida. A diferencia de los modelos lineales, las capas ocultas permiten aprender relaciones no lineales y combinaciones más complejas entre las características [23].

En la implementación se utilizaron capas densas, donde las neuronas de una capa se conectan con las neuronas de la siguiente. También se utilizaron funciones de activación ReLU y Dropout durante la experimentación para ayudar al modelo a aprender relaciones no lineales y disminuir el riesgo de sobreajuste.

El MLP se incorporó para determinar si una red neuronal densa puede identificar patrones más complejos entre las características utilizadas para clasificar el riesgo de fallecimiento hospitalario.

Red Neuronal Convolutiva 1D (CNN 1D)

La CNN 1D es una arquitectura de red neuronal que aplica filtros sobre una representación unidimensional de los datos para identificar patrones dentro de la entrada [24]. En este proyecto, el vector de características obtenido después del preprocesamiento se reorganizó para poder ser procesado por capas convolucionales unidimensionales.

La arquitectura también utiliza capas MaxPooling1D, cuya función es reducir progresivamente el tamaño de la representación conservando las activaciones más relevantes de cada región [25].

En este proyecto, la CNN 1D se evaluó como una arquitectura experimental aplicada a los datos tabulares preprocesados. Su finalidad fue comprobar si las operaciones convolucionales sobre la representación de las características podían ofrecer un mejor desempeño de clasificación frente a los modelos lineales y al MLP.

Red Neuronal Recurrente LSTM

La arquitectura Long Short-Term Memory (LSTM) es un tipo de red neuronal recurrente diseñada para procesar información organizada como una secuencia. Estas redes utilizan mecanismos internos de memoria que permiten conservar o descartar información durante el procesamiento [26].

Para utilizar esta arquitectura en el proyecto, las características preprocesadas se reorganizaron en una representación compatible con la entrada de una capa LSTM.

La LSTM se incorporó como una alternativa experimental para evaluar si el procesamiento recurrente de las características permitía mejorar la clasificación en comparación con la Regresión Logística, el Perceptrón, el MLP y la CNN 1D.

Base de datos para entrenamiento.

Campo	Descripción
Nombre del Dataset	Registro Estadístico de Egresos Hospitalarios - 2024
Fuente de origen	Portal Datos Abiertos Ecuador e Instituto Nacional de Estadística y Censos (INEC). La base contiene 1.132.667 registros y 38 variables sobre morbilidad y mortalidad hospitalaria reportada por establecimientos de salud con internación [7].

Mecanismo de recolección	La información proviene del “Informe Estadístico de Egresos Hospitalarios”, desarrollado en coordinación con el Ministerio de Salud Pública y el Instituto Ecuatoriano de Seguridad Social. Los datos son reportados por establecimientos de salud que prestan internación hospitalaria durante el año de estudio [7].
Organizaciones participantes	Instituto Nacional de Estadística y Censos (INEC), Ministerio de Salud Pública (MSP), Instituto Ecuatoriano de Seguridad Social (IESS) y establecimientos de salud públicos y privados con internación hospitalaria [7], [8].
Función en la solución	La base se utilizó para entrenar y evaluar los modelos de clasificación. Para la predicción se conservaron exclusivamente las variables disponibles al ingreso relacionadas con características demográficas, territoriales, temporales e institucionales. Las variables conocidas únicamente durante o al finalizar la hospitalización se excluyen del predictor para evitar fuga de información. La condición al egreso se utilizó únicamente como variable objetivo.

Componentes de la solución.

- **Módulo de extracción y preparación de datos:** Procesa el Registro Estadístico de Egresos Hospitalarios 2024, realiza la selección temporal de variables y elimina de las entradas predictivas la información conocida únicamente durante o después de la hospitalización.
- **Módulo de preprocesamiento e ingeniería de características:** Utiliza FeatureEngineer, SimpleImputer, StandardScaler, OneHotEncoder y

ColumnTransformer para transformar los registros originales en el vector de características utilizado por los modelos.

- **Módulo de entrenamiento y evaluación:** Realiza la partición de los datos, el tratamiento del desbalance mediante SMOTE y el entrenamiento de Regresión Logística, Perceptrón, MLP, CNN 1D y LSTM. Posteriormente compara su desempeño utilizando las métricas definidas para el proyecto.
- **Backend de inferencia:** Desarrollado mediante FastAPI, recibe los datos enviados por la interfaz, aplica el preprocesamiento y ejecuta el modelo seleccionado.
- **Interfaz web y visualización:** Permite ingresar los datos requeridos, consultar la estimación obtenida, acceder a estadísticas históricas y generar reportes.

Diseño de la interfaz gráfica

Predictor de Riesgo de Mortalidad Hospitalaria

Modelo de IA sobre datos de egresos hospitalarios

Predictor Estadísticas y Patrones Documentación

Datos del paciente e ingreso

Cargar datos de prueba

Cargar archivo JSON

El botón «Cargar datos de prueba» autocompleta el formulario con un caso clínico simulado (paciente femenina, 30 años).

PROVINCIA DEL ESTABLECIMIENTO

Elige o escribe...

CANTÓN DEL ESTABLECIMIENTO

Escribe el cantón...

ÁREA DEL ESTABLECIMIENTO

Elige o escribe...

CLASE DE ESTABLECIMIENTO

Elige o escribe...

TIPO DE ATENCIÓN

Elige o escribe...

ENTIDAD

Elige o escribe...

SECTOR

Elige o escribe...

NACIONALIDAD

Elige o escribe...

SEXO

Elige o escribe...

CÓDIGO DE EDAD

Elige o escribe...

EDAD

Elige o escribe...

ETNIA

Elige o escribe...

TIPO DE SEGURO

Elige o escribe...

DISCAPACIDAD DEL PACIENTE

Elige o escribe...

PROVINCIA DE RESIDENCIA

Elige o escribe...

CANTÓN DE RESIDENCIA

Escribe el cantón...

ÁREA DE RESIDENCIA

Elige o escribe...

MES DE INGRESO

Elige o escribe...

DÍA DE INGRESO

Elige o escribe...

FECHA DE INGRESO

mm/dd/yyyy



Predecir riesgo

Limpiar formulario

Resultado de Inferencia

Esperando datos...

Figura 8. Vista principal del formulario de ingreso de datos para la estimación del riesgo de fallecimiento hospitalario.

El módulo de Estadísticas y Patrones presenta información general obtenida del Registro Estadístico de Egresos Hospitalarios 2024. En la parte superior se muestran indicadores como el total de egresos analizados, la cantidad de registros correspondientes a pacientes vivos y fallecidos, la tasa de mortalidad observada y el Recall del modelo seleccionado.

1. Distribucion del Desemñace y Desbalance de Clases

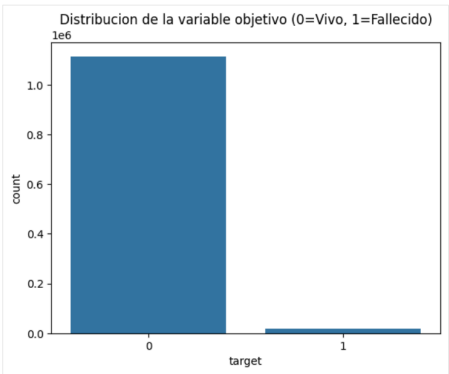
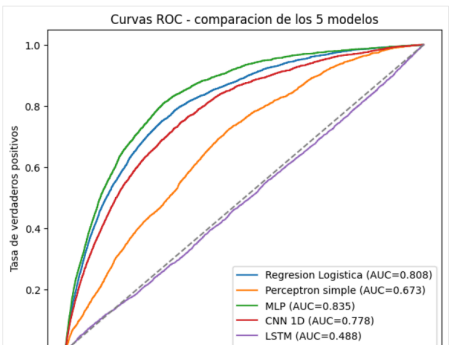


Figura 1: Proporción de egresos vivos (98.44%) vs fallecidos (1.56%) en el registro nacional 2024.

Hallazgos Epidemiológicos Clave

- **Desbalance Extremo (1:63):** Por cada paciente fallecido se registran aproximadamente 63 pacientes que egresan con vida.
- **Impacto en el Modelado de IA:** Modelos ingenuos que predicen "Vivo" para todos obtienen 98.44% de exactitud pero 0% de utilidad médica.
- **Tratamiento con SMOTE:** Se equilibró la distribución en entrenamiento a 891,954 casos por clase para forzar a los clasificadores a aprender patrones de riesgo crítico.

2. Evaluación y Curvas ROC Comparativas (5 Familias de IA)



Análisis del Desempeño de los Modelos

- **MLP (Red Neuronal Densa):** Obtuvo el mayor **ROC-AUC global (0.8351)** y mejor PR-AUC (0.0831), demostrando alta capacidad para ordenar el riesgo probabilístico.
- **Regresión Logística (Modelo Seleccionado):** Alcanzó el mayor **Recall (75.23%)** con un ROC-AUC sólido de **0.8078** y tiempo de respuesta inferior a 5 milisegundos.
- **Perceptrón Simple:** Logró un Recall del 37.60% y ROC-AUC de 0.6729.
- **CNN 1D y LSTM:** Tuvieron dificultades para converger en el espacio de características tabulares sin sobre-sesgarse.

Figura 10. Visualización de la distribución de la variable objetivo y del desbalance de clases.

Esta sección presenta gráficamente la distribución de la variable objetivo, diferenciando los egresos correspondientes a pacientes vivos y fallecidos. La visualización evidencia el fuerte desbalance del conjunto de datos, debido a que aproximadamente el 98.44 % de los registros corresponde a pacientes vivos y el 1.56 % a fallecimientos.

3. Tabla Oficial de Rendimiento Experimental

	modelo	recall	f1	auc_roc	pr_auc
0	Regresion Logistica	0.752327	0.079689	0.807822	0.069898
1	LSTM	0.598590	0.029833	0.488404	0.015389
2	MLP	0.538505	0.125687	0.835128	0.083107
3	Perceptron simple	0.376023	0.057296	0.672878	0.029852
4	CNN 1D	0.066008	0.081618	0.777571	0.057560

Tabla 1: Métricas de evaluación rigurosa sobre el 20% estratificado de test (INEC 2024).

4. Matrices de Confusión por Modelo

Visualización de aciertos, falsos positivos y falsos negativos evaluados en 226,534 casos de prueba:

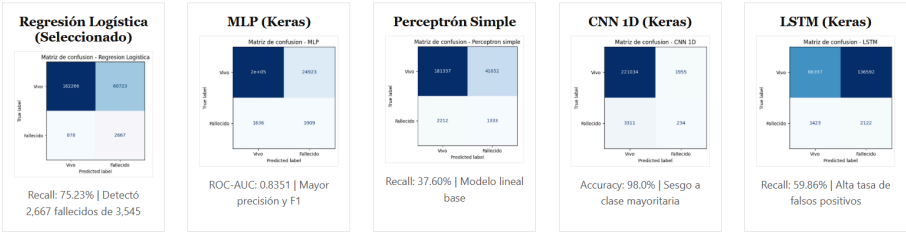


Figura 11. Comparación del rendimiento experimental de los cinco modelos de clasificación evaluados.

El módulo permite comparar los resultados obtenidos por Regresión Logística, Perceptrón Simple, MLP, CNN 1D y LSTM. Para ello se presentan las curvas ROC, métricas como Recall, F1-score, ROC-AUC y PR-AUC, y las matrices de confusión de cada clasificador.

Predictor de Riesgo de Mortalidad Hospitalaria

Modelo de Inteligencia Artificial sobre datos de egresos hospitalarios (INEC 2024)

Predictor Estadísticas y Patrones Documentación

Documentación y Guía del Usuario

Esta plataforma representa un sistema de apoyo a la decisión médica diseñado para estimar la **probabilidad de mortalidad intrahospitalaria** de un paciente. Las inferencias se realizan evaluando exclusivamente la información objetiva disponible al momento del ingreso al establecimiento de salud.

Nota Institucional: Este resultado es una estimación estadística basada en el registro oficial de 1,132,667 egresos del Instituto Nacional de Estadística y Censos (INEC) del año 2024. Su propósito es la priorización en triaje hospitalario y en ningún caso sustituye el criterio o diagnóstico de un profesional médico certificado.

Guía de Campos del Formulario de Ingreso

A continuación se detalla el significado, propósito y opciones de cada uno de los campos que se completan en el formulario de admisión:

Campo del Formulario	Opciones / Tipo de Dato	Significado y Utilidad en la Evaluación
Provincia del establecimiento	24 provincias del Ecuador	Provincia geográfica donde está ubicado el centro de salud. Refleja la capacidad y distribución de recursos hospitalarios por región.
Cantón del establecimiento	Cantones del Ecuador	Cantón específico donde se encuentra el hospital. Permite identificar centros con mayor concentración de especialistas e infraestructura.
Área del	Urbana / Rural	Diferencia entre hospitales en zonas urbanas con mayor

Figura 12. Vista del módulo de Documentación y guía de los campos utilizados por el sistema.

El módulo de Documentación proporciona información sobre el propósito del sistema, sus limitaciones y el significado de los campos utilizados durante la estimación. La guía describe cada variable requerida por el formulario, las opciones disponibles y su significado dentro del conjunto de datos.

Datos del paciente e ingreso

Cargar datos de prueba

Cargar archivo JSON

El botón «Cargar datos de prueba» autocompleta el formulario con un caso clínico simulado (paciente femenina, 30 años).

PROVINCIA DEL ESTABLECIMIENTO	CANTÓN DEL ESTABLECIMIENTO	ÁREA DEL ESTABLECIMIENTO
Loja	Loja	Rural
CLASE DE ESTABLECIMIENTO	TIPO DE ATENCIÓN	ENTIDAD
Hospital de especialidades	Crónico	Ministerio de Salud Pública
SECTOR	NACIONALIDAD	SEXO
Público	Ecuatoriano/a	Mujer
CÓDIGO DE EDAD	EDAD	ETNIA
Años (1 a 115 años de edad)	78	Mestizo/a
TIPO DE SEGURO	DISCAPACIDAD DEL PACIENTE	PROVINCIA DE RESIDENCIA
Ninguno	Física	Loja
CANTÓN DE RESIDENCIA	ÁREA DE RESIDENCIA	MES DE INGRESO
Loja	Rural	Agosto
DÍA DE INGRESO		
01		
FECHA DE INGRESO		
08/01/2026		
Formato enviado a la API: AAAA-MM-DD		

Predecir riesgo

Limpiar formulario

Figura 13. Ejemplo de formulario completado con datos de entrada para realizar una estimación.

La figura muestra un ejemplo de los datos cargados en el formulario antes de solicitar una estimación. La interfaz organiza las características en campos asociados con el establecimiento, el paciente, la residencia y la fecha de ingreso. Una vez completados y validados los datos, el usuario puede seleccionar la opción “Predecir riesgo”, enviando la información al backend para aplicar el preprocesamiento y ejecutar el modelo seleccionado.

Predicción generada con éxito.

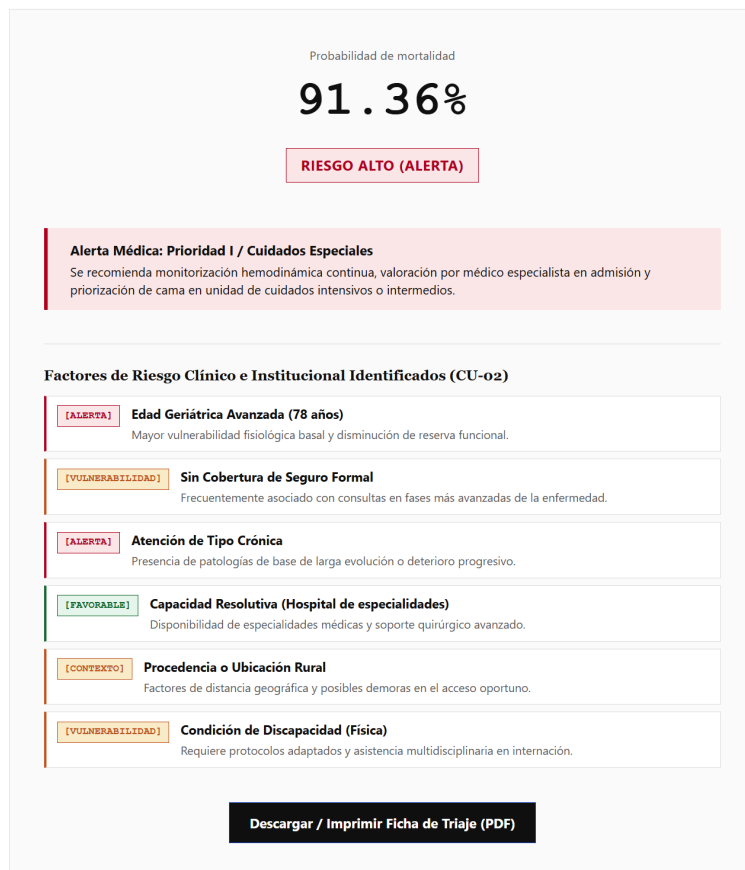


Figura 14. Vista del resultado generado por el modelo después de realizar una estimación.

Una vez procesados los datos, la interfaz presenta la probabilidad generada por el modelo y la clasificación obtenida de acuerdo con el umbral configurado. En el ejemplo mostrado se obtiene una probabilidad de 91.36 %, por lo que la interfaz presenta el resultado dentro de la categoría de riesgo alto. Adicionalmente, se muestran características asociadas con los datos ingresados para proporcionar contexto sobre la evaluación.

8:17:26, 6:56 PM

Predicador de Riesgo de Mortalidad Hospitalaria

Datos del paciente e ingreso

PROVINCIA DEL ESTABLECIMIENTO

Loja

CANTÓN DEL ESTABLECIMIENTO

Loja

ÁREA DEL ESTABLECIMIENTO

Rural

CLASE DE ESTABLECIMIENTO

Hospital de especialidades

TIPO DE ATENCIÓN

Crónico

ENTIDAD

Ministerio de Salud Pública

SECTOR

Público

NACIONALIDAD

Ecuatoriano/a

SEXO

Mujer

CÓDIGO DE EDAD

Años (1 a 115 años de edad)

EDAD

78

ETNIA

Mestizo/a

TIPO DE SEGURO

Ninguno

DISCAPACIDAD DEL PACIENTE

Física

PROVINCIA DE RESIDENCIA

Loja

CANTÓN DE RESIDENCIA

Loja

ÁREA DE RESIDENCIA

Rural

MES DE INGRESO

Agosto

DÍA DE INGRESO

01

FECHA DE INGRESO

08/01/2026

Formato enviado a la API: AAAA-MM-DD

Resultado de Inferencia

Predicción generada con éxito.

localhost:3000/index.html

1/3

Print

3 pages

Destination

Save as PDF

Pages

All

Layout

Landscape

More settings

Save

Cancel

Figura 15. Vista previa de la generación y exportación del reporte de una estimación en formato PDF.

La aplicación permite generar un reporte a partir de la información ingresada y del resultado obtenido durante la estimación. El usuario puede utilizar las funciones del navegador para guardar el documento en formato PDF o imprimirlo.

Implementación

El desarrollo de la solución se realizó principalmente utilizando Python para el procesamiento de datos, entrenamiento de modelos y construcción del backend. La interfaz de usuario se desarrolló utilizando tecnologías web. Para el entrenamiento de los modelos se utilizó Google Colab, permitiendo disponer de recursos de CPU y GPU para el procesamiento del conjunto completo de registros y para el entrenamiento de las arquitecturas de Deep Learning.

Las principales tecnologías utilizadas fueron:

- **Procesamiento y análisis de datos:** pandas y NumPy.
- **Machine Learning:** scikit-learn.

- **Tratamiento del desbalance:** imbalanced-learn mediante SMOTE [28].
- **Deep Learning:** TensorFlow, Keras y Keras Tuner.
- **Backend:** FastAPI, Uvicorn y Pydantic.
- **Frontend:** HTML5, CSS3 y JavaScript.
- **Persistencia de modelos:** Joblib para los componentes de scikit-learn y los mecanismos de serialización correspondientes para los modelos de Keras.
- **Entornos de desarrollo:** Google Colab, Jupyter Notebook y Visual Studio Code.

1. Carga y manipulación de datos

pandas: Para cargar, revisar y manipular la base de egresos hospitalarios en formato tabular, ya que es una librería orientada al análisis y manipulación de datos en Python [9].

numpy: Para operaciones numéricas y manejo de arreglos, debido a que es el paquete fundamental para computación científica en Python [10].

2. Preprocesamiento de datos

Para preparar los datos antes del entrenamiento se utilizó scikit-learn y imbalanced-learn siguiendo este flujo:

- `train_test_split`: Se dividieron los datos en 80% entrenamiento y 20% prueba con partición estratificada (`stratify=y`) para conservar la proporción del 1.56% de fallecimientos.
- `FeatureEngineer` (Transformador propio): Convirtió la edad multiescala (edad y `cod_edad`) a años decimales continuos (`edad_anios`) y extrajo el día de la semana de ingreso (`dow_ingr`).
- `ColumnTransformer`: Integró el tratamiento según el tipo de variable [11]:
 - Numéricas: Imputación por mediana con `SimpleImputer` y escalamiento con `StandardScaler`.
 - Categóricas: variables demográficas, territoriales e institucionales disponibles al ingreso; imputación por moda y codificación mediante `OneHotEncoder(handle_unknown='ignore')`, permite procesar nuevas categorías durante la inferencia sin provocar errores cuando una categoría específica no estuvo presente durante el entrenamiento. Después de aplicar las transformaciones, el vector utilizado por los modelos quedó compuesto por 182 características.

El modelo predictivo utiliza únicamente información disponible al momento del ingreso. Por lo tanto, se excluyeron de las entradas:

- Diagnóstico definitivo.
- Días totales de hospitalización.
- Especialidad de egreso.
- Fecha de egreso.
- Condición al egreso.

La condición al egreso se utilizó únicamente como variable objetivo para diferenciar los egresos vivos de los egresos con fallecimiento.

3. Partición de los datos y tratamiento del desbalance

El conjunto de datos se dividió en dos subconjuntos mediante `train_test_split`, utilizando una partición estratificada con `stratify=y`.

La distribución utilizada fue:

- 80 % para entrenamiento: 906,133 registros.
- 20 % para prueba: 226,534 registros.

Los registros correspondientes a fallecimiento representan aproximadamente 1.56 % del conjunto total, por lo que existe un fuerte desbalance entre las clases. Para reducir este problema durante el entrenamiento se utilizó SMOTE (Synthetic Minority Over-sampling Technique) sobre el conjunto de entrenamiento [27].

SMOTE genera ejemplos sintéticos de la clase minoritaria, permitiendo aumentar su representación durante el aprendizaje. Después de aplicar esta técnica, el conjunto destinado al entrenamiento alcanzó aproximadamente 1,783,908 registros balanceados. Se aplicó únicamente al conjunto de entrenamiento. El conjunto de prueba permaneció sin modificaciones para evaluar los modelos sobre datos que conservaran la distribución original.

4. Modelado de Machine Learning

Se implementaron cinco modelos de clasificación supervisada utilizando `scikit-learn` y `Keras/TensorFlow`. Se seleccionaron modelos con diferentes niveles de complejidad, desde

clasificadores lineales hasta arquitecturas de redes neuronales, con el propósito de comparar su capacidad para identificar los registros asociados con fallecimiento hospitalario.

1. **Regresión Logística:** utilizada como modelo lineal de referencia o *baseline*, permitiendo establecer un punto inicial de comparación frente a los modelos de mayor complejidad [12].
2. **Perceptrón Simple:** utilizado como un segundo enfoque de clasificación lineal, que ajusta sus pesos durante el entrenamiento a partir de los errores de clasificación [22].
3. **Multi-Layer Perceptron (MLP):** red neuronal densa implementada mediante Keras, utilizada para aprender relaciones no lineales y combinaciones más complejas entre las características de entrada [23].
4. **Red Neuronal Convolutiva 1D (CNN 1D):** arquitectura implementada mediante Keras que aplica operaciones convolucionales sobre la representación unidimensional de las características preprocesadas, con el objetivo de evaluar su capacidad para reconocer patrones útiles para la clasificación [24].
5. **Red Neuronal Recurrente LSTM:** arquitectura recurrente implementada mediante Keras que utiliza mecanismos internos de memoria para procesar la representación de las características y evaluar su desempeño frente a los demás modelos [26].

La comparación de estos cinco modelos permite evaluar tanto enfoques lineales como arquitecturas de Deep Learning y determinar experimentalmente cuál presenta el comportamiento más adecuado para el problema planteado.

5. Hiperparámetros y configuraciones finales

Los hiperparámetros determinan diferentes aspectos del proceso de aprendizaje y de la complejidad de cada modelo. Para los modelos tradicionales se evaluaron diferentes configuraciones mediante procesos de búsqueda y validación, mientras que para las arquitecturas de Deep Learning se utilizó Keras Tuner durante la experimentación. A partir de este proceso se establecieron las configuraciones finales presentadas en la siguiente tabla.

Las configuraciones utilizadas durante el entrenamiento final de los cinco clasificadores son las siguientes:

Modelo	Hiperparámetros principales	Optimización y criterio de parada
Regresión Logística	penalty = 'l2' • C = 1.0 • solver = 'lbfgs' • max_iter = 2000 • class_weight = 'balanced'	Optimización mediante L-BFGS y criterio de convergencia del algoritmo.
Perceptrón Simple	penalty = None • alpha = 0.0001 • max_iter = 1000 • tol = 1e-3 • class_weight = 'balanced'	Actualización iterativa de los pesos hasta alcanzar el criterio de convergencia o el máximo de iteraciones.
MLP	Entrada: 182 características • Dense(16, ReLU) • Dropout(0.2) • Dense(16, ReLU) • Dense(1, Sigmoid) • binary_crossentropy	Adam, learning_rate = 0.001, batch_size = 1024, máximo 60 épocas, EarlyStopping(patience=8) sobre val_loss.
CNN 1D	Entrada (182,1) • Conv1D(16, kernel=3) • MaxPooling • Dropout(0.1) • segundo bloque Conv1D • GlobalAveragePooling • Dense(64) • salida sigmoide	Adam, learning_rate = 0.001, batch_size = 1024, máximo 60 épocas, EarlyStopping(patience=8) sobre val_loss.
LSTM	Entrada (182,1) • LSTM(48) • Dense(32, ReLU) • Dense(1, Sigmoid)	Adam, learning_rate = 0.001, batch_size = 1024, máximo 60 épocas, EarlyStopping(patience=8) sobre val_loss.

Tabla Oficial de Hiperparámetros y Configuraciones

6. Justificación técnica del entrenamiento

Para los modelos MLP, CNN 1D y LSTM se estableció un máximo de 60 épocas y un tamaño de lote (batch_size) de 1024 muestras. Debido al tamaño del conjunto balanceado mediante SMOTE, cada época procesa una gran cantidad de ejemplos y realiza múltiples actualizaciones de los parámetros de la red.

Para reducir el riesgo de sobreajuste se utilizó EarlyStopping con una paciencia de 8 épocas, monitoreando val_loss. De esta manera, el entrenamiento podía finalizar anticipadamente cuando el desempeño sobre los datos de validación dejaba de mejorar.

En las redes neuronales se utilizó binary_crossentropy como función de pérdida, debido a que el problema corresponde a una clasificación binaria entre las clases vivo y fallecido.

Justificación de las arquitecturas

El MLP se configuró utilizando dos capas densas de 16 neuronas con activación ReLU. Además, se incorporó Dropout de 0.2 como mecanismo de regularización. Esta configuración permitió evaluar relaciones no lineales mediante una estructura relativamente compacta frente al vector de 182 características.

La CNN 1D recibió las características en una representación de dimensión (182, 1) y aplicó filtros convolucionales para evaluar experimentalmente si las transformaciones locales sobre el vector podían mejorar la clasificación.

La LSTM utilizó 48 unidades recurrentes seguidas de una capa densa de 32 neuronas. Su incorporación permitió comparar una arquitectura recurrente frente a los modelos lineales, el MLP y la CNN 1D.

Entre las arquitecturas de Deep Learning, el MLP presentó el mejor rendimiento general, obteniendo un ROC-AUC de 0.8351, PR-AUC de 0.0831 y F1-score de 0.1257.

7. Selección del modelo para la aplicación

Aunque el MLP presentó los mejores valores de ROC-AUC, PR-AUC y F1-score, el criterio establecido para la integración del modelo priorizó la detección de los registros correspondientes a fallecimiento.

La Regresión Logística Balanceada alcanzó un Recall de 75.23 %, el mayor entre los cinco modelos evaluados. Además, obtuvo un ROC-AUC de 0.8078.

Por este motivo, la Regresión Logística fue seleccionada como modelo final para ser integrada en la aplicación. Esta decisión no significa que la Regresión Logística haya sido superior en todas las métricas. El MLP presentó un mejor comportamiento general en diferentes indicadores, pero la Regresión Logística resultó más adecuada según el criterio de Recall.

8. Umbral de clasificación

La Regresión Logística genera una probabilidad continua asociada con la clase de fallecimiento. Para convertir esta probabilidad en una clasificación binaria se utilizó un umbral de 0.50.

Cuando la probabilidad estimada es igual o superior al 50 %, el sistema clasifica el resultado como riesgo alto. Cuando la probabilidad se encuentra por debajo del umbral, el resultado se clasifica como riesgo inferior al umbral establecido.

9. Guardado del modelo

joblib: Para guardar el modelo entrenado sin necesidad de volver a entrenarlo desde cero [15].

Descripción de los Resultados

Métricas de evaluación

Para evaluar los cinco modelos implementados se utilizaron métricas de clasificación en el conjunto de prueba que permitieron analizar el rendimiento global y la detección de la clase minoritaria [6], [14]:

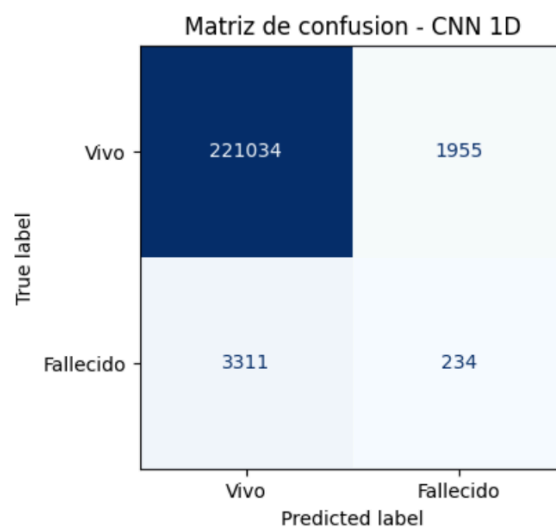
- Exactitud (Accuracy): Proporción total de predicciones correctas.
- Precisión: Proporción de predicciones de fallecimiento que fueron correctas.
- Sensibilidad o Recall: Proporción de fallecimientos reales detectados por el modelo.
- F1-Score: Media armónica entre precisión y sensibilidad.
- ROC-AUC y PR-AUC: Capacidad de discriminación de clases y rendimiento en la clase minoritaria.
- Matriz de confusión: Cuantificación exacta de verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos.

Criterio de selección: Debido al fuerte desbalance entre las clases, el modelo final no se seleccionó únicamente mediante Accuracy. Para el objetivo del proyecto se prioriza especialmente el Recall de la clase fallecimiento, con el propósito de reducir la cantidad de casos positivos no detectados. Esta métrica se complementa con Precision, F1-score, ROC-AUC y PR-AUC. La Regresión Logística obtuvo el mayor Recall entre los cinco modelos, con 75.23 %, junto con un ROC-AUC de 0.8078, por lo que fue seleccionada para su integración en la aplicación.

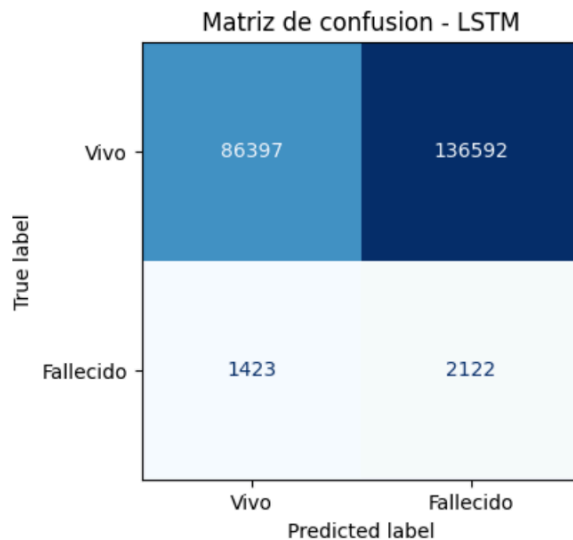
Métricas de evaluación obtenidas

Se utilizaron matrices de confusión para analizar con mayor detalle los aciertos y errores producidos por cada modelo sobre el conjunto de prueba. Estas matrices permiten distinguir los registros correctamente clasificados y los diferentes tipos de error cometidos por cada clasificador.

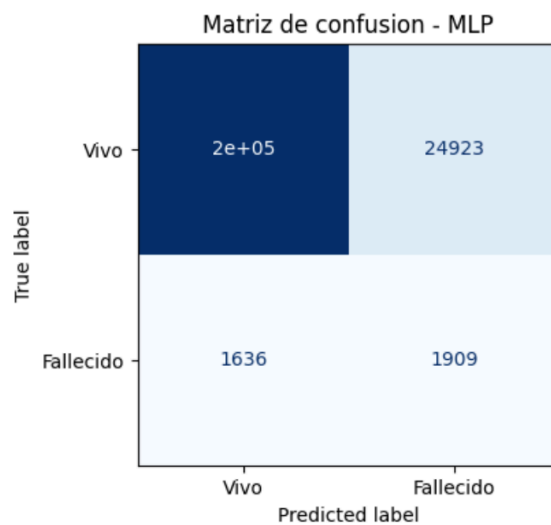
En las matrices, las filas representan la clase real y las columnas la clase predicha. La clase Vivo corresponde a la clase negativa y Fallecido a la clase positiva.



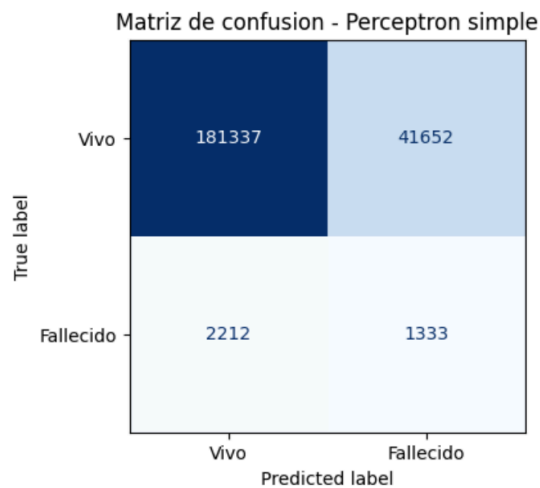
La CNN 1D clasificó correctamente la gran mayoría de los registros correspondientes a pacientes vivos, con 221,034 verdaderos negativos. Sin embargo, solamente identificó correctamente 234 casos de fallecimiento, mientras que 3,311 fallecimientos fueron clasificados como vivos. Este comportamiento explica por qué el modelo puede alcanzar un Accuracy elevado y, al mismo tiempo, presentar un Recall muy bajo para la clase minoritaria. El resultado evidencia la importancia de no utilizar Accuracy como único criterio de selección en conjuntos fuertemente desbalanceados.



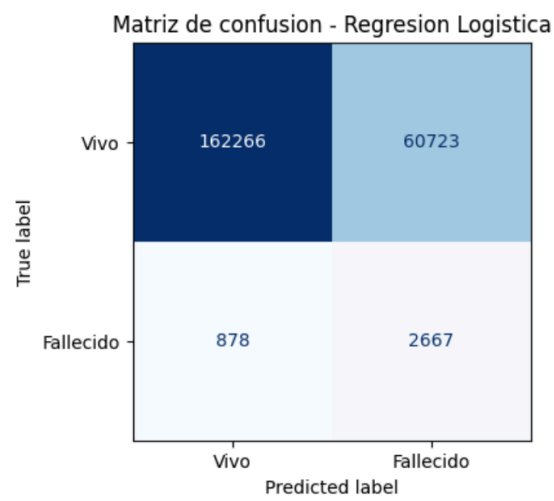
La LSTM identificó correctamente 2,122 casos de fallecimiento y presentó 1,423 falsos negativos. Sin embargo, clasificó una cantidad muy elevada de pacientes vivos como fallecidos, generando 136,592 falsos positivos.



El MLP presentó un comportamiento más equilibrado entre la identificación de ambas clases. Detectó correctamente 1,909 fallecimientos, mientras que 1,636 casos de fallecimiento fueron clasificados como vivos. Produce menos falsos positivos que modelos como Regresión Logística y LSTM.



El Perceptrón Simple identificó correctamente 1,333 registros de fallecimiento, mientras que 2,212 fallecimientos fueron clasificados como vivos. Su comportamiento fue inferior al de Regresión Logística y MLP para la detección de la clase minoritaria, mostrando una mayor cantidad de falsos negativos.



La Regresión Logística identificó correctamente 2,667 de los 3,545 registros correspondientes a fallecimiento, mientras que 878 casos fueron clasificados incorrectamente como vivos. También produjo una cantidad considerable de falsos positivos, evidenciando el compromiso existente entre aumentar la sensibilidad y mantener una mayor precisión.

Evaluación de los modelos

Modelo	Accuracy	Precisión	Recall	F1	ROC-AUC	PR-AUC

Regresión Logística	73.18 %	4.25 %	75.23 %	0.0805	0.8078	0.0699
MLP	88.75 %	7.08 %	54.44 %	0.1257	0.8351	0.0831
Perceptrón	85.58 %	4.07 %	37.60 %	0.0734	0.6729	0.0385
LSTM	39.11 %	1.54 %	59.86 %	0.0300	0.5085	0.0163
CNN 1D	98.01 %	5.34 %	6.60 %	0.0591	0.5255	0.0195

Métricas de Rendimiento Obtenidas sobre el Conjunto de Prueba (226,534 Registros)

Selección del modelo final

La CNN 1D obtuvo el Accuracy más alto, con 98.01 %. Sin embargo, su Recall para la clase fallecimiento fue únicamente de 6.60 %. Este resultado demuestra por qué Accuracy puede ser engañoso en un conjunto fuertemente desbalanceado. Un modelo puede clasificar correctamente gran parte de la clase mayoritaria y, al mismo tiempo, detectar muy pocos casos pertenecientes a la clase minoritaria.

El MLP presentó el mejor desempeño general en diferentes métricas, obteniendo el mayor ROC-AUC (0.8351), PR-AUC (0.0831), Precision (7.08 %) y F1-score (0.1257). Sin embargo, su Recall fue de 54.44 %, inferior al obtenido por la Regresión Logística.

La LSTM alcanzó un Recall de 59.86 %, pero presentó un ROC-AUC cercano a 0.5.

El Perceptrón obtuvo resultados intermedios, con 37.60 % de Recall y ROC-AUC de 0.6729.

Finalmente, la Regresión Logística alcanzó el mayor Recall de los cinco modelos, con 75.23 %, junto con un ROC-AUC de 0.8078. Debido al criterio para priorizar la detección de la clase fallecimiento, este modelo fue el seleccionado.

Contribución en el desarrollo de la solución

Leonardo Zambrano

- Desarrollo del Backend: Creación de la API en FastAPI para recibir los datos del paciente y enviar las predicciones en tiempo real.
- Integración del Modelo: Implementación de la carga en memoria de los archivos entrenados (.joblib) para que el servidor responda de forma instantánea.
- Validación de Datos: Programación de las reglas para verificar que los datos del formulario lleguen completos y sin errores antes de calcular el riesgo.
- Preprocesamiento: Creación de la función para unificar la edad (horas, días, meses, años) a un solo formato numérico.

Javier Murillo

- Diseño de la Interfaz Web: Creación de las páginas web del sistema (index.html para el formulario, stats.html para las estadísticas y about.html para la documentación). Y estilo con HTML5 Y CSS.
- Módulo de Reportes: Programación del botón para imprimir y descargar la Ficha de Triage en formato PDF.
- Entrenamiento de Modelos Lineales: Implementación y ajuste de los modelos base de Machine Learning (Regresión Logística y Perceptrón Simple).
- Análisis de Matrices de Confusión: Evaluación de los aciertos y falsos positivos de los modelos lineales para verificar la detección de pacientes críticos.

Joel Guamaní

- Preparación de Datos: Descarga, limpieza y filtrado de la base de datos del INEC 2024 (1.13 millones de registros), asegurando usar únicamente datos conocidos al ingreso.
- Entrenamiento de los Modelos: Implementación de los 5 algoritmos de Inteligencia Artificial (Regresión Logística, Perceptrón, MLP, CNN 1D y LSTM) en Google Colab.
- Balanceo de Clases: Aplicación de la técnica SMOTE para equilibrar los casos de pacientes fallecidos y vivos durante el entrenamiento.
- Evaluación de Resultados: Cálculo de las métricas de rendimiento (Recall, ROC-AUC, Accuracy) y generación de las curvas y matrices de confusión.

Conclusiones e Impacto

1. Prevención de fuga de información y preparación de los datos.

En nuestro proyecto debimos delimitar correctamente las variables disponibles al momento del ingreso y excluir información conocida únicamente durante o después de la hospitalización, como el diagnóstico definitivo, los días de estadía y la condición al egreso. Esta separación permitió evitar *data leakage*. Además, mediante FeatureEngineer se unificó la edad registrada en diferentes unidades y se generaron nuevas características temporales, obteniendo una representación de 182 características después del preprocesamiento.

2. Tratamiento del desbalance y selección adecuada de métricas.

El conjunto de datos presentó un fuerte desbalance, con aproximadamente 1.56 % de fallecimientos frente a 98.44 % de egresos vivos. Por esta razón se utilizó SMOTE sobre el conjunto de entrenamiento y se consideraron métricas adicionales al Accuracy. Los resultados demostraron la importancia de esta decisión: la CNN 1D alcanzó un Accuracy de 98.01 %, pero solamente un Recall de 6.60 % para la clase fallecimiento. Confirmando que en este problema, métricas como Recall, F1-score, ROC-AUC y PR-AUC resultan más informativas para comparar los modelos.

3. La mayor complejidad del modelo no garantizó mejores resultados.

La comparación entre los modelos permitió comprobar que las arquitecturas más complejas no necesariamente producen el mejor comportamiento sobre este tipo de datos. El MLP obtuvo el mejor desempeño general en ROC-AUC (0.8351), PR-AUC y F1-score; sin embargo, la Regresión Logística alcanzó el mayor Recall, con 75.23 %, junto con un ROC-AUC de 0.8078. Esto demuestra la importancia de comparar modelos de distinta complejidad antes de seleccionar una solución final.

4. La Regresión Logística resultó la alternativa más adecuada para el criterio priorizado.

La Regresión Logística fue seleccionada como modelo final debido a que identificó 2,667 de los 3,545 fallecimientos presentes en el conjunto de prueba, obteniendo el mayor Recall entre los cinco clasificadores evaluados. La Selección se realizó priorizando la detección de la clase minoritaria. Además, su menor complejidad facilita la interpretación de su funcionamiento frente a arquitecturas de Deep Learning.

5. El proyecto permitió integrar el modelo en una solución funcional de extremo a extremo.

El trabajo no se limitó al entrenamiento de modelos en notebooks, sino que se integró el preprocesamiento, el modelo seleccionado, un backend desarrollado con FastAPI y una interfaz web. La aplicación permite ingresar datos, realizar estimaciones, consultar estadísticas históricas, revisar información del sistema y generar reportes. Nuestro proyecto demuestra cómo un modelo de Machine Learning puede pasar a ser una solución de software siendo una herramienta académica de apoyo y no como sustituto del criterio médico.

Referencias

[1] Instituto Nacional de Estadística y Censos. “Registro Estadístico de Egresos Hospitalarios - 2024”. Disponible en:

<https://www.datosabiertos.gob.ec/dataset/registro-estadistico-de-egresos-hospitalarios-2024>

[2] Instituto Nacional de Estadística y Censos. “Camas y Egresos Hospitalarios”. Ecuador en Cifras. Disponible en:

<https://www.ecuadorencifras.gob.ec/camas-y-egresos-hospitalarios/>

[3] Instituto Nacional de Estadística y Censos. “Anuario de Camas y Egresos Hospitalarios”. Ecuador en Cifras. Disponible en:

<https://www.ecuadorencifras.gob.ec/anuario-de-camas-y-egresos-hospitalarios/>

[4] Scikit-learn. “1.1. Linear Models”. Disponible en:

https://scikit-learn.org/stable/modules/linear_model.html

[5] IBM. “Multilayer Perceptron”. IBM SPSS Statistics Documentation. Disponible en:

<https://www.ibm.com/docs/en/spss-statistics/30.0.0?topic=edition-multilayer-perceptron>

[6] Scikit-learn. “sklearn.metrics”. Disponible en:

<https://scikit-learn.org/stable/api/sklearn.metrics.html>

[7] Instituto Nacional de Estadística y Censos. “Registro Estadístico de Egresos Hospitalarios - 2024”. Datos Abiertos Ecuador. Disponible en:

<https://www.datosabiertos.gob.ec/dataset/registro-estadistico-de-egresos-hospitalarios-2024>

[8] Instituto Nacional de Estadística y Censos. “Camas y Egresos Hospitalarios”. Ecuador en Cifras. Disponible en:

<https://www.ecuadorencifras.gob.ec/camas-y-egresos-hospitalarios/>

[9] pandas, “pandas - Python Data Analysis Library.” Disponible en:

<https://pandas.pydata.org/>

[10] NumPy Developers, “NumPy documentation.” Disponible en:

<https://numpy.org/doc/stable/>

[11] scikit-learn developers, “ColumnTransformer.” Disponible en:

<https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html>

[12] scikit-learn developers, “LogisticRegression.” Disponible en:

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

[13] scikit-learn developers, “MLPClassifier.” Disponible en:

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

[14] scikit-learn developers, “Metrics and scoring: quantifying the quality of predictions.”

Disponible en: https://scikit-learn.org/stable/modules/model_evaluation.html

[15] joblib developers, “joblib.dump.” Disponible en:

<https://joblib.readthedocs.io/en/latest/generated/joblib.dump.html>

[16] scikit-learn developers. “DecisionTreeClassifier”. Disponible en:

<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>

[17] scikit-learn developers. “RandomForestClassifier”. Disponible en:

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

[18] scikit-learn developers. “HistGradientBoostingClassifier”. Disponible en:

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html>

[19] scikit-learn developers, “Pipeline”. Disponible en:

<https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html>

[20] scikit-learn developers, “SimpleImputer”. Disponible en:

<https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html>

[21] scikit-learn developers, “RandomizedSearchCV”. Disponible en:

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html

[22] scikit-learn developers, “Perceptron”. Disponible en:

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Perceptron.html

[23] Keras developers, “Dense layer”. Keras 3 API Documentation. Disponible en:

https://keras.io/api/layers/core_layers/dense/

[24] Keras developers, “Conv1D layer”. Keras 3 API Documentation. Disponible en:

https://keras.io/api/layers/convolution_layers/convolution1d/

[25] Keras developers, “MaxPooling1D layer”. Keras 3 API Documentation. Disponible en:

https://keras.io/api/layers/pooling_layers/max_pooling1d/

[26] Keras developers, “LSTM layer”. Keras 3 API Documentation. Disponible en:

https://keras.io/api/layers/recurrent_layers/lstm/

[27] N. V. Chawla, K. W. Bowyer, L. O. Hall y W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002. Disponible en: <https://doi.org/10.1613/jair.953>

[28] imbalanced-learn developers, “SMOTE.” Disponible en:

https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html

Guía para probar Interfaz

Para probar la aplicación se deben seguir los siguientes pasos:

1. Descomprimir el proyecto y abrir una terminal en la carpeta principal.
2. Crear un entorno virtual de Python: `python -m venv .venv`
3. Activar el entorno virtual.

En Windows: `.venv\Scripts\activate`

En Linux/macOS: `source .venv/bin/activate`

4. Instalar las dependencias del proyecto utilizando el archivo `requirements.txt`:

```
pip install -r requirements.txt
```

5. Iniciar el backend de inferencia. Desde la carpeta `backend/` se ejecuta:

```
cd backend
```

```
uvicorn main:app --reload
```

El servicio queda disponible de manera local. Puede verificarse su funcionamiento accediendo al endpoint de salud definido por la aplicación o mediante la documentación automática proporcionada por FastAPI.

6. Iniciar la interfaz web en una segunda terminal. Desde la carpeta `frontend/` puede levantarse un servidor HTTP local:

```
cd frontend
```

```
python -m http.server 5500
```

Luego se accede desde el navegador a: `http://localhost:5500`

Declaración de librerías no públicas

En este proyecto no se utilizó ninguna librería privada, propietaria ni de pago. Todas las herramientas y librerías empleadas para el procesamiento de datos, entrenamiento de los modelos, construcción del backend y ejecución del sistema son de código abierto y acceso público.

Las dependencias utilizadas pueden instalarse mediante los mecanismos estándar del ecosistema de Python y se encuentran especificadas en el archivo requirements.txt. Por esta razón, para preparar el entorno de ejecución únicamente es necesario utilizar:

```
pip install -r requirements.txt
```

Esto significa que no se requiere adquirir licencias comerciales, disponer de claves de activación ni acceder a repositorios privados para ejecutar el proyecto. Librerías como pandas, NumPy, scikit-learn, imbalanced-learn, TensorFlow, Keras, FastAPI, Uvicorn, Pydantic y Joblib se encuentran disponibles públicamente.

Conjunto de ejemplo para correr el ejemplo

Es posible comprobar directamente el funcionamiento del backend mediante una solicitud HTTP desde la terminal. El endpoint utilizado por el sistema es:

```
POST /api/v1/predict/riesgo-mortalidad
```

El esquema PacienteInput recibe las 20 variables de ingreso utilizadas por la aplicación.

Un ejemplo de solicitud mediante cURL es:

```
curl -X POST "http://127.0.0.1:8000/api/v1/predict/riesgo-mortalidad" \  
-H "Content-Type: application/json" \  
-d '{  
  "prov_ubi": "Loja",  
  "cant_ubi": "Loja",  
  "area_ubi": "Rural",
```

"clase": "Hospital de especialidades",
"tipo": "Crónico",
"entidad": "Ministerio de Salud Pública",
"sector": "Público",
"nac_pac": "Ecuatoriano/a",
"sexo": "Mujer",
"cod_edad": "Años",
"edad": 78,
"etnia": "Mestizo/a",
"tipo_seg": "Ninguno",
"dis_pac": "Física",
"prov_res": "Loja",
"cant_res": "Loja",
"area_res": "Rural",
"mes_ingr": "Agosto",
"dia_ingr": 1,
"fecha_ingr": "2026-08-01"
}